

Figure 7.3 Acceptance tests that fail too often can lead to complacency.

It's an important issue, and it's worth putting in some effort up front to ensure that the time you invest automating your acceptance criteria yields its full fruits over the duration of the project and beyond. Automated tests are like any other kind of code—if you want them to be robust and easy to maintain, you need to put a little effort into their design and write them in a way that's clear and easy to understand. Writing good automated acceptance tests requires the same software engineering skills and disciplines, and the same level of craftsmanship, as well-written production code. The best automated acceptance tests tend to be the result of a collaborative effort between testers and developers.

Unfortunately, automated tests often don't receive the attention they need. Part of this problem is historical—for many years, test automation has been synonymous with "test scripts." Teams have been reluctant to put too much effort into writing scripts when they could be writing production code. Many of the more traditional testing tools do indeed use scripting languages that have poor support for, or simply don't encourage, standard software engineering practices, such as refactoring to avoid code duplication, writing reusable components, and writing code in a clean and readable manner.

Many of these issues can be avoided if teams apply a few simple principles. A good automated acceptance test needs to respect a few key rules:

- *It should communicate clearly.* Automated acceptance tests are first and foremost communication tools. Wording and semantics are important: the tests need to clearly explain the business behavior they're demonstrating. They need to explain to stakeholders, business analysts, testers, and other team members what business tasks the application is intended to support, and illustrate how it does so. Ideally, these tests will outlive the development phase of the project and go on to help the maintenance or BAU[1] team understand the project requirements and how they've been implemented.

[1] Business as Usual—the team that takes care of applications once they're deployed into production. In many organizations, this is a different team than the one developing the project.